



兰州工业学院

实 验 报 告

(2023 —2024 学年第 1 学期)

课程名称 面向对象程序设计

专业班级 专升本电信 23

学 号 ZB2305010116

姓 名 姜 勇

实验项目	实验一 类的定义和对象创建		
实验时间	2023 年 9 月 20 日	实验地点	信息与通信工程专业实验室 (DSP)
一、实验目的 1. 掌握类、类的数据成员、类的成员函数的定义方式。 2. 理解类成员的访问控制方式。 3. 掌握对象的定义和操作对象的方法。 4. 掌握重载构造函数的方法,理解构造函数的执行过程。 二、实验环境 硬件：计算机一台； 软件：VC++ 6.0 或 Code::Blocks。 三、实验内容 1. 请定义一个矩形类(Rectangle)，私有数据成员为矩形的长度(len)和宽度(wid)，缺省构造函数置 len 和 wid 为 0，有参构造函数置 len 和 wid 为对应形参的值，另外还包括求矩形周长、求矩形面积、取矩形长度和宽度、修改矩形长度和宽度为对应形参的值、输出矩形尺寸等公有成员函数。要求输出矩形尺寸的格式为“length: 长度, width: 宽度”。编写主函数对定义类进行测试。 矩形类 <pre> #ifndef RECTANGLE_H #define RECTANGLE_H #include <iostream> using namespace std; class Rectangle { public: //缺省构造函数 Rectangle() { len = 0; wid = 0; } // 有参构造函数 </pre>			

```

    Rectangle(double l, double w)
    {
        len = l;
        wid = w;
    }
    //设置长和宽
    void setlen(double l);
    void setwid(double w);
    //获取长宽
    double getlen();
    double getwid();
    //获取周长和面积
    double Perimeter();
    double Area();
    // 输出矩形尺寸
    void printSize()
    {
        cout << "length: " << len << ", width: " << wid << endl;
    }
private:
    double len;
    double wid;
};

#endif // RECTANGLE_H
矩形类成员的实现
#include "Rectangle.h"

//成员函数的定义
//设置矩形长宽的方法

```

```

void Rectangle::setlen(double l) {
    len = l;
}

void Rectangle::setwid(double w) {
    wid = w;
}

//获取矩形长宽的方法

double Rectangle::getlen() {
    return len;
}

double Rectangle::getwid() {
    return wid;
}

//设置矩形周长面积的方法

double Rectangle::Perimeter() {
    return 2*(len+wid);
}

double Rectangle::Area() {
    return len*wid;
}

主函数

#include <iostream>

#include "Rectangle.h"

using namespace std;

int main()
{
    Rectangle rectangle1; // 使用缺省构造函数创建矩形对象
    rectangle1.printSize(); // 输出矩形尺寸
}

```

象

```
Rectangle rectangle2(4.5, 2.3); // 使用有参构造函数创建矩形对象

rectangle2.printSize(); // 输出矩形尺寸

cout << "Perimeter: " << rectangle2.Perimeter() << endl;

cout << "Area: " << rectangle2.Area() << endl;

//修改之后的值

cout << "修改矩形长度和宽度" << endl;

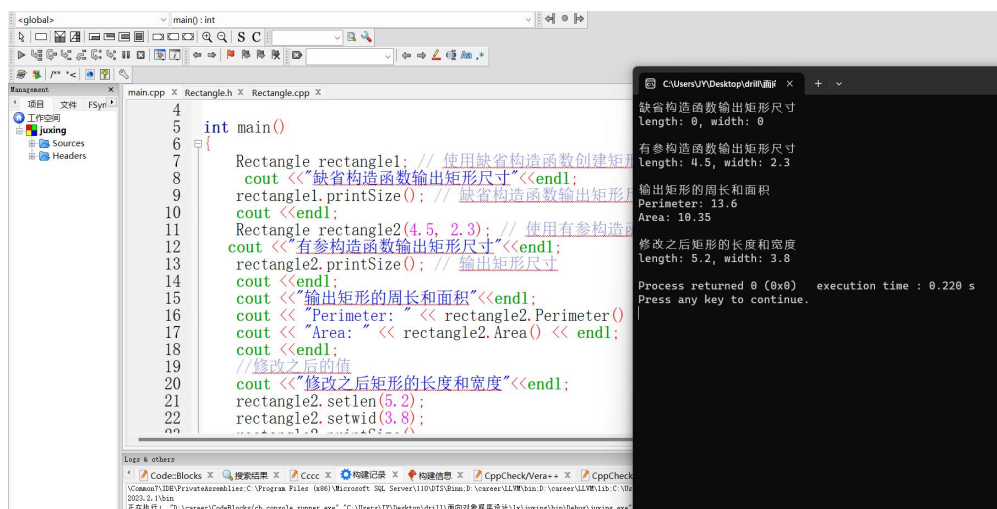
rectangle2.setlen(5.2);

rectangle2.setwid(3.8);

rectangle2.printSize();

return 0;

}
```



```
int main()
{
    Rectangle rectangle1; // 使用缺省构造函数创建矩形对象
    cout << "缺省构造函数输出矩形尺寸" << endl;
    rectangle1.printSize(); // 缺省构造函数输出矩形尺寸
    cout << endl;
    Rectangle rectangle2(4.5, 2.3); // 使用有参构造函数创建矩形对象
    cout << "有参构造函数输出矩形尺寸" << endl;
    rectangle2.printSize(); // 输出矩形尺寸
    cout << endl;
    cout << "输出矩形的周长和面积" << endl;
    cout << "Perimeter: " << rectangle2.Perimeter() << endl;
    cout << "Area: " << rectangle2.Area() << endl;
    cout << endl;
    // 修改之后的值
    cout << "修改之后矩形的长度和宽度" << endl;
    rectangle2.setlen(5.2);
    rectangle2.setwid(3.8);
    rectangle2.printSize();
    return 0;
}
```

输出结果:

```
缺省构造函数输出矩形尺寸
length: 0, width: 0
有参构造函数输出矩形尺寸
length: 4.5, width: 2.3
输出矩形的周长和面积
Perimeter: 13.6
Area: 10.35
修改之后矩形的长度和宽度
length: 5.2, width: 3.8
Process returned 0 (0x0)   execution time : 0.220 s
Press any key to continue.
```

图一 输出缺省构造函数、有参构造函数的矩形尺寸，矩形的周长面积，修改后的长度宽度

2. 定义一个学生类，包含 6 个成员变量，用来保存学生的姓名、年龄、学号和三门功课成绩；设计三个以上的构造函数；设计三个以上的成员函数，完成对 6 个成员变量的操作，要求年龄的数据范围为 16~30，成绩的数据范围为 0~100. 编写主函数对定义的类进行测试。

学生类

```
#ifndef STUDENT_H

#define STUDENT_H

#include <iostream>
```

```

#include <string>

class student
{
public:
    // 默认构造函数
    student();

    // 构造函数 1, 初始化学生姓名、年龄和学号
    student(const std::string& studentName, int studentAge, const
std::string& id);

    // 构造函数 2, 初始化学生姓名、年龄、学号和三门功课成绩
    student(const std::string& studentName, int studentAge, const
std::string& id, double s1, double s2, double s3);

    // 设置并获取年龄, 限制数据范围为 16~30
    void setAge(int studentAge);
    int getAge() const;

    // 设置并获取三门功课成绩, 限制数据范围为 0~100
    void setScore1(double s1);
    double getScore1() const;
    void setScore2(double s2);
    double getScore2() const;
    void setScore3(double s3);
    double getScore3() const;

    // 输出学生信息
    void printInfo() const;

private:
    std::string name;

    int age;

```

```

        std::string studentId;

        double score1;

        double score2;

        double score3;
    };
#endif // STUDENT_H

学生类成员的实现

#include "student.h"

// 默认构造函数
student::student()
{
    name = "";
    age = 0;
    studentId = "";
    score1 = 0.0;
    score2 = 0.0;
    score3 = 0.0;
}

// 构造函数 1, 初始化学生姓名、年龄和学号
student::student(const std::string& studentName, int studentAge,
const std::string& id)
{
    name = studentName;
    setAge(studentAge);
    studentId = id;
    score1 = 0.0;
    score2 = 0.0;

```

```

        score3 = 0.0;
    }

    // 构造函数 2，初始化学生姓名、年龄、学号和三门功课成绩
    student::student(const std::string& studentName, int studentAge,
const std::string& id, double s1, double s2, double s3)
    {
        name = studentName;
        setAge(studentAge);
        studentId = id;
        setScore1(s1);
        setScore2(s2);
        setScore3(s3);
    }

    // 设置年龄，限制数据范围为 16~30
    void student::setAge(int studentAge)
    {
        if (studentAge >= 16 && studentAge <= 30)
        {
            age = studentAge;
        }
        else
        {
            std::cout << "年龄的范围为 16~30，请重新输入" << std::endl;
        }
    }

    int student::getAge() const
    {

```



```

        return age;
    }

    // 设置三门功课成绩，限制数据范围为 0~100，并获取
    void student::setScore1(double s1)
    {
        if (s1 >= 0 && s1 <= 100)
        {
            score1 = s1;
        }
        else
        {
            std::cout << "成绩的范围为 0~100，请重新输入" << std::endl;
        }
    }

    double student::getScore1() const
    {
        return score1;
    }

    void student::setScore2(double s2)
    {
        if (s2 >= 0 && s2 <= 100)
        {
            score2 = s2;
        }
        else
        {
            std::cout << "成绩的范围为 0~100，请重新输入" << std::endl;
        }
    }

```

```

    }

}

double student::getScore2() const
{
    return score2;
}

void student::setScore3(double s3)
{
    if (s3 >= 0 && s3 <= 100)
    {
        score3 = s3;
    }
    else
    {
        std::cout << "成绩的范围为 0~100, 请重新输入" << std::endl;
    }
}

double student::getScore3() const
{
    return score3;
}

// 输出学生信息
void student::printInfo() const
{
    std::cout << "姓名: " << name << std::endl;
    std::cout << "年龄: " << age << std::endl;
    std::cout << "学号: " << studentId << std::endl;
}

```

```

        std::cout << "成绩 1: " << score1 << std::endl;
        std::cout << "成绩 2: " << score2 << std::endl;
        std::cout << "成绩 3: " << score3 << std::endl;
    }

```

主函数

```

#include <iostream>

#include "student.h"

using namespace std;

int main()
{

```

 // 创建并初始化学生对象 student1，指定学生的姓名、年龄、学号
和三门功课的成绩

```

        student student1("JY", 22, "ZB2305010116", 95.5, 92.5, 94.0);
        // 调用 printInfo() 方法打印学生信息
        student1.printInfo();
        cout << endl;

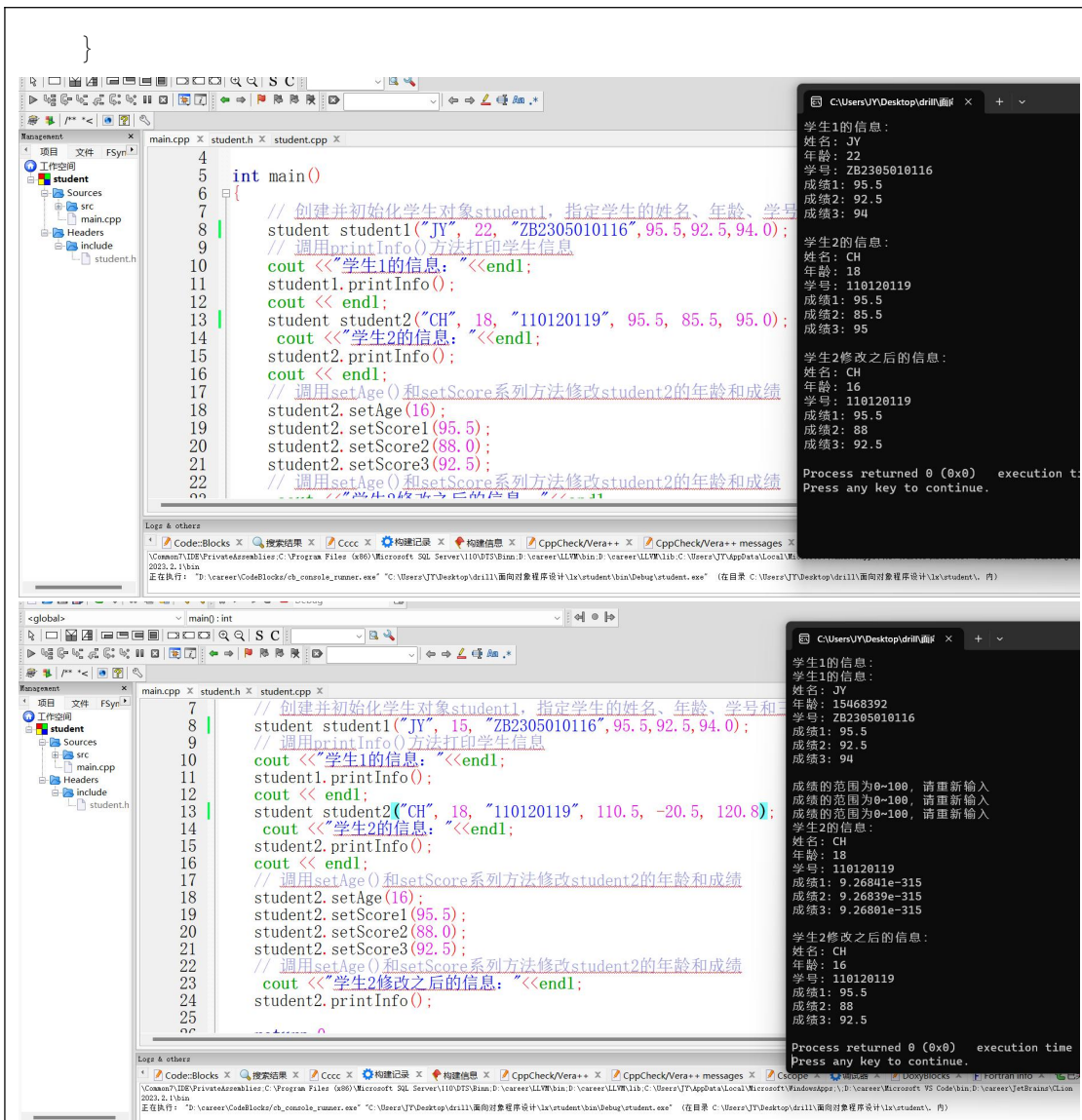
        student student2("CH", 18, "110120119", 90.5, 85.5, 95.0);
        student2.printInfo();
        cout << endl;

        // 调用 setAge() 和 setScore 系列方法修改 student2 的年龄和成绩
        student2.setAge(16);
        student2.setScore1(95.5);
        student2.setScore2(88.0);
        student2.setScore3(92.5);

        // 调用 setAge() 和 setScore 系列方法修改 student2 的年龄和成绩
        student2.printInfo();

        return 0;

```



年龄和成绩超出规定范围的运行结果

3. 自拟题目。

定义了一个工资类，包含四个成员变量，设计了三个以上的构造函数以及三个以上的成员函数，并编写了一个主函数来测试该类。

工资类

```

#ifndef SALARY_H
#define SALARY_H
#include <string>

class Salary {

```

```

private:

    std::string name; //姓名

    double jbgz; //基本工资

    double bonus; //奖金

    double pay; //津贴

public:

    // 构造函数

    Salary();

    //初始化对象

    Salary(const std::string& n, double basic);

    Salary(const std::string& n, double basic, double b, double a);

    // 成员函数

    //设置奖金和津贴

    void setBonus(double b);

    void setpay(double a);

    //计算总工资

    double sum() const;

    //打印工资详情

    void printSalaryDetails() const;

    //获取姓名

    std::string getName() const;

};

```

```

#endif //SALARY_H

```

工资类中成员的实现

```

#include "salary.h"

#include <iostream>

//将这些都设置为默认值

```

```

Salary::Salary() {
    name = ""; //姓名
    jbgz = 0.0; //基本工资
    bonus = 0.0; //奖金
    pay = 0.0; //津贴
}

//接受姓名和基本工资作为参数，并将奖金和津贴设置为默认值。
Salary::Salary(const std::string& n, double basic) {
    name = n;
    jbgz = basic;
    bonus = 0.0;
    pay = 0.0;
}

//接受姓名、基本工资、奖金和津贴作为参数
Salary::Salary(const std::string& n, double basic, double b, double
a) {
    name = n;
    jbgz = basic;
    bonus = b;
    pay = a;
}

//设置奖金和津贴。
void Salary::setBonus(double b) {
    bonus = b;
}

void Salary::setpay(double a) {
    pay = a;
}

```

```

}

//计算总工资。

double Salary::sum() const {

    return jbgz + bonus + pay;

}

//打印工资详情

void Salary::printSalaryDetails() const {

    std::cout << "姓名: " << name << std::endl;
    std::cout << "基本工资: " << jbgz << std::endl;
    std::cout << "奖金: " << bonus << std::endl;
    std::cout << "津贴: " << pay << std::endl;
    std::cout << "工资总额: " << sum() << std::endl;

}

//获取姓名, 返回姓名

std::string Salary::getName() const {

    return name;

}

```

主函数

```

#include <iostream>

#include "Salary.h"

int main() {

    // 使用不同的构造函数创建工资对象

    Salary employee1; // 默认构造函数

    Salary employee2("姜勇", 3500.0); // 构造函数 1

    Salary employee3("李", 3000.0, 500.0, 200.0); // 构造函数 2

    // 设置员工 1 的奖金和津贴

    employee1.setBonus(1000.0);
}

```

```

employee1.setpay(150.0);

// 打印所有员工的工资详情

std::cout << "员工 1:" << std::endl;

employee1.printSalaryDetails();

std::cout << std::endl;

std::cout << "员工 2:" << std::endl;

employee2.printSalaryDetails();

std::cout << std::endl;

std::cout << "员工 3:" << std::endl;

employee3.printSalaryDetails();

return 0;

}

```

The screenshot shows a C++ IDE with the following code in the main.cpp file:

```

4 int main() {
5     // 使用不同的构造函数创建工资对象
6     Salary employee1; // 默认构造函数
7     Salary employee2("姜勇", 3500.0); // 构造函数1
8     Salary employee3("李", 3000.0, 500.0, 200.0); // 构造函数2
9
10    // 设置员工1的奖金和津贴
11    employee1.setBonus(1000.0);
12    employee1.setpay(150.0);
13
14    // 打印所有员工的工资详情
15    std::cout << "员工1:" << std::endl;
16    employee1.printSalaryDetails();
17    std::cout << std::endl;
18
19    std::cout << "员工2:" << std::endl;
20    employee2.printSalaryDetails();
21    std::cout << std::endl;
22
23    std::cout << "员工3:" << std::endl;
24    employee3.printSalaryDetails();
25
26    return 0;
27 }

```

The output window shows the following results:

```

员工1:
姓名:
基本工资: 0
奖金: 1000
津贴: 150
工资总额: 1150

员工2:
姓名: 姜勇
基本工资: 3500
奖金: 0
津贴: 0
工资总额: 3500

员工3:
姓名: 李
基本工资: 3000
奖金: 500
津贴: 200
工资总额: 3700

Process returned 0 (0x0)   execution time : 0.016 s
Press any key to continue.

```

四、实验总结

通过本次实验，我明白了类的定义是将相关的数据和函数封装到一个实体中，它可以规范化数据的访问和操作，并将逻辑相关的代码组织在一起，提高代码的可读性和可维护性。成员变量代表对象的状态或属性，成员函数定义了对对象的行为。通过在类中定义和实现成员函数，可以将逻辑相关的代码集中到一起，提高代码的清晰度和复用性。动态分配对象可以使用‘new’运算符，它返回指向对象的指针。但在使用完后要记得使用‘delete’运算符手动释放

内存，以防止内存泄漏。

函数重载在同一个作用域内，可以声明几个功能类似的同名函数，但是这些同名函数的形式参数必须不同。

实验一成绩评定表

序号	考核项目	分值分布	成绩
1	出勤与纪律情况	10	
2	实验任务完成情况	40	
3	实验报告质量	50	
总分			
指导教师签字			